

Movie Recommendation System using Machine Learning

AI & Machine Learning Internship Project Report

Submitted By

N. Karthik

Internship

Elevate Labs – AI & Machine Learning Internship

Abstract

The Movie Recommendation System is a web-based application developed using Python, Streamlit, and Machine Learning techniques. The system recommends movies similar to a user's selected movie by analyzing movie descriptions, genres, and language. It uses TF-IDF (Term Frequency–Inverse Document Frequency) and Cosine Similarity to identify movies with similar content. The application provides an interactive interface where users can search movies, filter by language and genre, and receive personalized recommendations.

1. Introduction

With thousands of movies available across multiple platforms, finding relevant content has become challenging. Recommendation systems help users discover movies based on their interests.

This project implements a content-based movie recommendation system that analyzes movie descriptions and recommends movies with similar characteristics. The application features a modern Streamlit interface and supports English, Hindi, and Telugu movies.

2. Problem Statement

Users often spend significant time searching for movies that match their preferences. Traditional search methods rely only on titles or categories and fail to provide personalized recommendations.

The proposed system addresses this problem by analyzing movie content and recommending similar movies using Machine Learning algorithms.

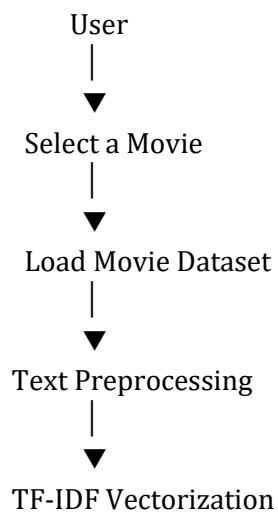
3. Objectives

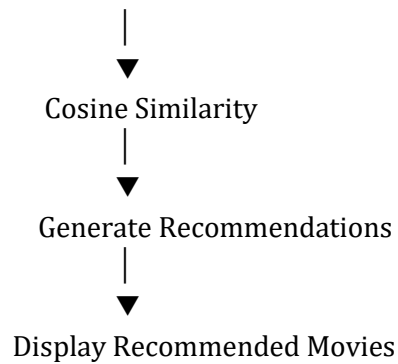
- Recommend movies based on similarity.
- Build an interactive web application.
- Demonstrate content-based recommendation techniques.
- Provide filtering by language and genre.
- Improve movie discovery experience.

4. Technologies Used

Technology	Purpose
Python	Programming Language
Streamlit	Web Application Framework
Pandas	Dataset Handling
Scikit-learn	Machine Learning
TF-IDF Vectorizer	Feature Extraction
Cosine Similarity	Recommendation Algorithm
HTML & CSS	UI Styling

5. System Architecture





6. Methodology

Step 1: Load Dataset

The application loads the movie dataset containing movie titles, genres, descriptions, languages, and ratings.

Step 2: Data Preprocessing

Movie descriptions, genres, and language information are combined into a single text field.

Step 3: Feature Extraction

TF-IDF Vectorizer converts textual information into numerical feature vectors.

Step 4: Similarity Calculation

Cosine Similarity calculates similarity between movies.

Step 5: Recommendation Generation

The system identifies the movies with the highest similarity scores and recommends them.

Step 6: Display Results

Recommended movies are displayed with:

- Match Percentage
 - Movie Rating
 - Genre
 - Language
-

7. Features

- Movie Recommendation
 - Search Movies
 - Language Filter
 - Genre Filter
 - Rating Filter
 - Interactive Dashboard
 - Similarity Percentage
 - Modern User Interface
 - Responsive Design
-

8. Project Structure

```
movie_recommender/  
|  
├─ app.py  
├─ movies_data.py  
├─ bg.jpg  
├─ requirements.txt  
├─ README.md  
└─ screenshots/
```

9. Machine Learning Algorithms

TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF converts movie descriptions into numerical vectors by assigning importance to words based on their frequency.

Advantages

- Efficient
 - Lightweight
 - Works well for text-based recommendation
-

Cosine Similarity

Cosine Similarity measures the angle between two TF-IDF vectors to determine how similar two movies are.

Similarity Score:

- **100%** → Highly Similar
 - **70-99%** → Very Similar
 - **50-69%** → Moderately Similar
 - **Below 50%** → Less Similar
-

10. Workflow

1. Load movie dataset.
 2. Preprocess movie information.
 3. Create TF-IDF vectors.
 4. Calculate cosine similarity.
 5. Select the highest similarity scores.
 6. Recommend similar movies.
 7. Display movie details.
-

11. Results

The application successfully:

- Recommended similar movies.
- Displayed similarity percentages.
- Filtered movies by language.
- Filtered movies by genre.
- Displayed movie ratings.
- Provided a clean and interactive interface.

12. Advantages

- Fast recommendations.
- Easy to use.
- Interactive interface.
- Supports multiple languages.
- Efficient recommendation algorithm.
- No user login required.
- Lightweight application.

13. Limitations

- Uses content-based filtering only.
- Recommendations depend on available movie descriptions.
- Does not learn user preferences over time.
- Limited to the movies available in the dataset.

14. Future Enhancements

- Integration with IMDb API.
- Display movie posters.
- Trailer support.
- User authentication.
- Watchlist feature.
- Personalized recommendations.
- Collaborative filtering.
- Deep Learning-based recommendation engine.
- Real-time database integration.

15. Conclusion

The Movie Recommendation System successfully demonstrates the practical application of Machine Learning in entertainment. By combining TF-IDF Vectorization and Cosine Similarity, the application provides fast and accurate content-based movie recommendations. The interactive Streamlit interface makes it user-friendly, while filtering options enhance the overall experience. This project effectively showcases recommendation system concepts and can be extended with advanced personalization techniques in the future.